Game-Playing Agent with Hidden Information

```
import sys, subprocess, getpass, random, itertools, json
subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "-U", "google-genai"])

from google import genai
from collections import defaultdict, Counter

API_KEY = getpass.getpass("Enter Gemini API Key: ")
client = genai.Client(api_key=API_KEY)
MODEL = "gemini-3.6-flash"

SUITS = ["♠", "♥", "♦", "♣"]
RANKS = list(range(2, 15))
RANK_NAMES = {11:"J", 12:"Q", 13:"K", 14:"A"}

def card_name(card):
    return str(RANK_NAMES.get(card[0], card[0])) + card[1]

def create_deck():
    deck = [(r, s) for r in RANKS for s in SUITS]
    random.shuffle(deck)
    return deck

def evaluate_hand(cards):
    values = sorted([c[0] for c in cards], reverse=True)
    counts = Counter(values)
    suits = [c[1] for c in cards]
    flush = len(set(suits)) == 1
    unique = sorted(set(values), reverse=True)
    straight_high = 0
    if 14 in unique:
        unique.append(1)
    for i in range(len(unique)-4):
        if unique[i] - unique[i+4] == 4:
            straight_high = unique[i]
            break
    groups = sorted([(count, rank) for rank, count in counts.items()], reverse=True)

    if flush and straight_high:
        return (8, straight_high)
    if groups[0][0] == 4:
        return (7, groups[0][1], groups[1][1])
    if groups[0][0] == 3 and groups[1][0] == 2:
        return (6, groups[0][1], groups[1][1])
    if flush:
        return (5, *values)
    if straight_high:
        return (4, straight_high)
    if groups[0][0] == 3:
        return (3, groups[0][1], *sorted([r for r in values if r != groups[0][1]], reverse=True))
    if groups[0][0] == 2 and groups[1][0] == 2:
        pairs = sorted([g[1] for g in groups if g[0] == 2], reverse=True)
        kicker = max(r for r in values if r not in pairs)
        return (2, pairs[0], pairs[1], kicker)
    if groups[0][0] == 2:
        return (1, groups[0][1], *sorted([r for r in values if r != groups[0][1]], reverse=True))
    return (0, *values)

def estimate_win_probability(hole, community, opponents, simulations=1200):
    wins = 0
    ties = 0
    for _ in range(simulations):
        deck = [c for c in create_deck() if c not in hole + community]
        random.shuffle(deck)
        board = community + deck[:5-len(community)]
        index = 5-len(community)
        opponent_hands = [
            deck[index+i*2:index+(i+1)*2]
            for i in range(opponents)
        ]
        my_score = evaluate_hand(hole + board)
        opponent_scores = [evaluate_hand(hand + board) for hand in opponent_hands]
        best_opponent = max(opponent_scores)
        if my_score > best_opponent:
            wins += 1
        elif my_score == best_opponent:
            ties += 1
    return round((wins + ties*0.5)/simulations, 3)
```

```
class GamePlayingAgent:
    def __init__(self):
        self.history = defaultdict(list)
        self.opponent_profiles = defaultdict(lambda: {
            "fold": 0, "call": 0, "raise": 0
        })
        self.results = []
        self.total_wins = 0
        self.total_games = 0


    def observe(self, player, action):
        self.history[player].append(action)
        self.opponent_profiles[player][action.lower()] += 1


    def predict_opponent(self, player):
        profile = self.opponent_profiles[player]
        total = sum(profile.values())
        if total == 0:
            return "Unknown"
        return max(profile, key=profile.get)

    def choose_action(self, probability, current_bet, chips):
        if probability >= 0.70:
            return "RAISE"
        elif probability >= 0.40:
            return "CALL" if current_bet <= chips * 0.15 else "FOLD"
        elif probability >= 0.20:
            return "CALL" if current_bet <= chips * 0.05 else "FOLD"
        return "FOLD"

    def gemini_analysis(self, game_state):
        prompt = f"""
        You are a strategic poker analysis assistant.
        Analyze this simplified four-player poker game.
        Game state:
        {json.dumps(game_state, indent=2)}
        Explain:
        1. Strength of the player's hand.
        2. Opponent betting behavior.
        3. Uncertainty and possible risks.
        4. Whether the recommended action is reasonable.
        5. A short strategic recommendation.
        The mathematical win probability and recommended action are already calculated.
        Do not change them.
        """
        response = client.models.generate_content(
            model=MODEL,
            contents=prompt
        )
        return response.text

    def update_result(self, won):
        self.total_games += 1
        if won:
            self.total_wins += 1
        self.results.append("WIN" if won else "LOSS")

    def report(self):
        print("\n========== LONG-TERM PERFORMANCE REPORT ==========")
        print("Total Games:", self.total_games)
        print("Total Wins:", self.total_wins)
        print("Total Losses:", self.total_games-self.total_wins)
        print("Win Rate:", round(self.total_wins/max(1,self.total_games)*100,2), "%")
        print("\nOpponent Action Histories:")
        for player, actions in self.history.items():
            print(player, actions)
        print("\nOpponent Behavior Profiles:")
        for player in self.opponent_profiles:
            print(player, dict(self.opponent_profiles[player]))
        print("\nGame Results:", self.results)

def play_game(agent, round_number):
    deck = create_deck()
    player_hand = deck[:2]
    community = deck[2:7]
    opponents = ["Player 2", "Player 3", "Player 4"]

    print("\n" + "="*55)
    print("GAME ROUND", round_number)
    print("="*55)
```

```
    print("Your Private Cards:", [card_name(c) for c in player_hand])
    print("Community Cards:", [card_name(c) for c in community[:3]])

    betting_actions = {}
    for player in opponents:
        action = random.choices(
            ["FOLD", "CALL", "RAISE"],
            weights=[0.25, 0.55, 0.20],
            k=1
        )[0]
        betting_actions[player] = action
        agent.observe(player, action)

    print("\nOpponent Actions:")
    for player, action in betting_actions.items():
        print(player, ":", action, "| Predicted behavior:", agent.predict_opponent(player))

    probability = estimate_win_probability(
        player_hand,
        community[:3],
        len(opponents)
    )

    current_bet = random.choice([10, 20, 50, 100])
    chips = 1000
    action = agent.choose_action(probability, current_bet, chips)

    print("\nEstimated Win Probability:", probability*100, "%")
    print("Current Bet:", current_bet)
    print("Available Chips:", chips)
    print("AI Recommended Action:", action)

    game_state = {
        "round": round_number,
        "private_cards": [card_name(c) for c in player_hand],
        "community_cards": [card_name(c) for c in community[:3]],
        "opponent_actions": betting_actions,
        "opponent_predictions": {
            p: agent.predict_opponent(p) for p in opponents
        },
        "win_probability": probability,
        "recommended_action": action,
        "current_bet": current_bet
    }

    print("\n========== GEMINI STRATEGIC ANALYSIS ==========")
    try:
        print(agent.gemini_analysis(game_state))
    except Exception as e:
        print("Gemini analysis unavailable:", str(e))

    print("\n========== SHOWDOWN ==========")
    print("Final Community Cards:", [card_name(c) for c in community])

    my_score = evaluate_hand(player_hand + community)
    opponent_hands = [deck[7+i*2:9+i*2] for i in range(3)]
    opponent_scores = [evaluate_hand(h + community) for h in opponent_hands]

    best_score = max([my_score] + opponent_scores)
    won = my_score == best_score

    if won:
        print("Result: YOU WIN")
    else:
        print("Result: YOU LOSE")

    print("Your Hand Score:", my_score)
    print("Best Opponent Score:", max(opponent_scores))

    agent.update_result(won)

agent = GamePlayingAgent()

number_of_rounds = int(input("Enter number of game rounds [5]: ") or 5)

for round_number in range(1, number_of_rounds+1):
    play_game(agent, round_number)

agent.report()
```

```
Estimated Win Probability: 25.7 %
Current Bet: 20
Available Chips: 1000
AI Recommended Action: CALL

========== GEMINI STRATEGIC ANALYSIS ==========
Here is the strategic analysis for the current game state:

### 1. Strength of the Player's Hand
* **Current Hand:** Pair of Sevens (7♠ in hand + 7♦ on the flop) with a 6 kicker.
* **Board Texture:** The flop is `7♦ J♠ 10♥`.
* **Evaluation:** You currently hold a weak bottom pair on a coordinated board containing high cards (Jack, Ten

### 2. Opponent Betting Behavior
* **Player 2 & Player 4:** Both folded, leaving only Player 3 in the hand with you.
* **Player 3:** Called the current bet of 20. This indicates Player 3 either has a marginal hand (e.g., a Ten, v

### 3. Uncertainty and Possible Risks
* **Low Win Probability:** Your win chance is **25.7%**.
* **Board Hazards:** Player 3 could easily hold a Jack (top pair), a Ten (better mid pair), or straight draws tl
* **Kicker Problem:** Even if you hit a two-pair (a 6 on the turn or river), your kicker remains weak against ma

### 4. Feasibility of the Recommended Action ("CALL")
* **Reasonableness:** **Yes, the recommended action to CALL (20) is reasonable.**
* **Justification:** You are now head-to-head against a single opponent. The cost to see the Turn is relatively

### 5. Short Strategic Recommendation
**Call the 20 bet to see the Turn card.**
* If the Turn improves your hand (e.g., another Spade, a 7, or a 6), re-evaluate for potential value or continue
* If the Turn is a blank and Player 3 bets aggressively, look to fold and cut your losses, as your pair of 7s w

========== SHOWDOWN ==========
Final Community Cards: ['7♦', 'J♠', '10♥', '2♦', 'A♣']
Result: YOU LOSE
Your Hand Score: (1, 7, 14, 11, 10, 6, 2)
Best Opponent Score: (1, 12, 14, 11, 10, 7, 2)

======================================================
GAME ROUND 2
======================================================
Your Private Cards: ['9♠', 'A♦']
Community Cards: ['3♣', '10♠', '3♥']

Opponent Actions:
Player 2 : CALL | Predicted behavior: fold
Player 3 : CALL | Predicted behavior: call
Player 4 : CALL | Predicted behavior: fold

Estimated Win Probability: 21.9 %
Current Bet: 20
Available Chips: 1000
AI Recommended Action: CALL

========== GEMINI STRATEGIC ANALYSIS ==========
Gemini analysis unavailable: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently exper
```

Video Incident-Review Agent

```
import sys, subprocess, os, json, cv2, base64, getpass, shutil
subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "-U", "google-genai", "opencv-python-headle

from google import genai
from google.genai import types
from google.colab import files
from IPython.display import display, Image
from collections import Counter

API_KEY = getpass.getpass("Enter Gemini API Key: ")
client = genai.Client(api_key=API_KEY)
MODEL = "gemini-3.6-flash"

uploaded = files.upload()
video_path = next(iter(uploaded.keys()))

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    raise ValueError("Unable to open video file")

fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
duration = total_frames / fps if fps > 0 else 0

print("Video:", video_path)
print("Duration:", round(duration, 2), "seconds")
print("FPS:", fps)
```

```
print("Total Frames:", total_frames)

interval = int(input("Extract one frame every how many seconds? [3]: ") or 3)
frame_interval = max(1, int(fps * interval))
output_folder = "incident_evidence"
os.makedirs(output_folder, exist_ok=True)

frames = []
frame_number = 0

while True:
    success, frame = cap.read()
    if not success:
        break

    if frame_number % frame_interval == 0:
        timestamp = frame_number / fps if fps > 0 else 0
        height, width = frame.shape[:2]
        resized = cv2.resize(frame, (min(1024, width), int(height * min(1024, width) / width)))
        filename = os.path.join(output_folder, f"frame_{int(timestamp)}s.jpg")
        cv2.imwrite(filename, resized)

        frames.append({
            "timestamp": round(timestamp, 2),
            "path": filename
        })

    frame_number += 1

cap.release()

print("\nExtracted Frames:", len(frames))

incidents = []

for i, frame_info in enumerate(frames):
    image_file = frame_info["path"]

    with open(image_file, "rb") as f:
        image_data = f.read()

    prompt = """
    You are a video incident-review agent analyzing a surveillance frame.

    Examine the image for visible safety-related incidents:
    - Collision or possible collision
    - Near miss
    - Unauthorized entry indicators
    - Unsafe movement
    - Fall or injury
    - Dangerous proximity to machinery
    - Other visible safety hazards

    Do not invent events that cannot be established from this single frame.
    Do not identify people by identity.
    If no clear incident is visible, return incident_detected as false.

    Return only valid JSON:
    {
      "incident_detected": true,
      "incident_type": "incident category",
      "severity": "Low/Medium/High",
      "description": "Visible evidence and explanation",
      "objects_involved": ["object1", "object2"],
      "recommended_action": "Suggested safety action",
      "confidence": 0.0
    }
    """

    try:
        response = client.models.generate_content(
            model=MODEL,
            contents=[
                types.Part.from_bytes(data=image_data, mime_type="image/jpeg"),
                prompt
            ],
            config=types.GenerateContentConfig(
                response_mime_type="application/json",
                temperature=0.2
            )
        )
```

```
            result = json.loads(response.text)

            if result.get("incident_detected", False):
                result["timestamp_seconds"] = frame_info["timestamp"]
                result["evidence_frame"] = image_file
                incidents.append(result)

            print("Processed frame", i+1, "/", len(frames))

        except Exception as e:
            print("Frame processing error:", str(e))

    report = {
        "video_file": video_path,
        "video_duration_seconds": round(duration, 2),
        "frames_extracted": len(frames),
        "total_incidents": len(incidents),
        "incidents": incidents
    }

    severity_counts = Counter(i.get("severity", "Unknown") for i in incidents)

    print("\n========== INCIDENT REVIEW REPORT ==========")
    print(json.dumps(report, indent=4))

    print("\n========== INCIDENT SUMMARY ==========")
    print("Total Incidents:", len(incidents))
    print("High Severity:", severity_counts["High"])
    print("Medium Severity:", severity_counts["Medium"])
    print("Low Severity:", severity_counts["Low"])

    for incident in incidents:
        print("\nTimestamp:", incident["timestamp_seconds"], "seconds")
        print("Type:", incident.get("incident_type"))
        print("Severity:", incident.get("severity"))
        print("Description:", incident.get("description"))
        print("Confidence:", incident.get("confidence"))
        print("Recommended Action:", incident.get("recommended_action"))
        print("Evidence:")
        display(Image(filename=incident["evidence_frame"]))

    report_path = "incident_review_report.json"
    with open(report_path, "w") as f:
        json.dump(report, f, indent=4)

    print("\nReport saved:", report_path)

    shutil.make_archive("incident_review_evidence", "zip", output_folder)

    files.download(report_path)
    files.download("incident_review_evidence.zip")
```

```
Enter Gemini API Key: ·····
Choose files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 12901485-uhd_3840_2160_24fps.mp4 to 12901485-uhd_3840_2160_24fps.mp4
Video: 12901485-uhd_3840_2160_24fps.mp4
Duration: 11.59 seconds
FPS: 23.976023976023978
Total Frames: 278
Extract one frame every how many seconds? [3]: 2

Extracted Frames: 6
Processed frame 1 / 6
Frame processing error: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently experienci
Frame processing error: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently experienci
Frame processing error: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently experienci
Frame processing error: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently experienci
Frame processing error: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently experienci

========== INCIDENT REVIEW REPORT ==========
{
    "video_file": "12901485-uhd_3840_2160_24fps.mp4",
    "video_duration_seconds": 11.59,
    "frames_extracted": 6,
    "total_incidents": 1,
    "incidents": [
        {
            "incident_detected": true,
            "incident_type": "Vehicle Collision",
            "severity": "High",
            "description": "A red tractor-trailer semi-truck has been involved in a severe crash on a multi-lane
            "objects_involved": [
                "semi-truck",
                "fluid spill",
                "debris",
                "highway"
            ],
            "recommended_action": "Dispatch emergency responders, police traffic control, and hazardous spill c
            "confidence": 0.98,
            "timestamp_seconds": 0.0,
            "evidence_frame": "incident_evidence/frame_0s.jpg"
        }
    ]
}

========== INCIDENT SUMMARY ==========
Total Incidents: 1
High Severity: 1
Medium Severity: 0
Low Severity: 0

Timestamp: 0.0 seconds
Type: Vehicle Collision
Severity: High
Description: A red tractor-trailer semi-truck has been involved in a severe crash on a multi-lane highway, caus
Confidence: 0.98
Recommended Action: Dispatch emergency responders, police traffic control, and hazardous spill cleanup crews im
Evidence:
```



Wikipedia Search Agent

```
!pip install -q -U google-genai requests

import requests
import json
from google import genai
from getpass import getpass

API_KEY = getpass("Enter your Gemini API Key: ")
client = genai.Client(api_key=API_KEY)

MODEL = "gemini-3.5-flash-lite"

session = requests.Session()
session.headers.update({
    "User-Agent": "WikipediaSearchAgent/1.0 (educational project)"
})

def gemini(prompt):
    response = client.models.generate_content(
        model=MODEL,
        contents=prompt
    )
    return response.text

def wikipedia_search(query):
    url = "https://en.wikipedia.org/w/api.php"
```

```
        params = {
            "action": "query",
            "list": "search",
            "srsearch": query,
            "srlimit": 5,
            "format": "json"
        }

        response = session.get(url, params=params, timeout=20)
        response.raise_for_status()

        results = response.json()["query"]["search"]

        return [
            {
                "title": item["title"],
                "snippet": item["snippet"]
            }
            for item in results
        ]

    def get_article(title):
        url = "https://en.wikipedia.org/w/api.php"

        params = {
            "action": "query",
            "prop": "extracts|info",
            "explaintext": 1,
            "exintro": 0,
            "exchars": 12000,
            "inprop": "url",
            "titles": title,
            "format": "json"
        }

        response = session.get(url, params=params, timeout=20)
        response.raise_for_status()

        pages = response.json()["query"]["pages"]
        page = next(iter(pages.values()))

        return {
            "title": page.get("title", title),
            "content": page.get("extract", ""),
            "url": page.get("fullurl", "")
        }

    class WikipediaSearchAgent:

        def __init__(self):
            self.history = []

        def run(self, question):

            print("\nAgent: Understanding your query...")

            search_prompt = f"""
            You are a Wikipedia search agent.
            Identify 2 to 3 effective Wikipedia search queries
            for the following question.

            Question: {question}

            Return only valid JSON:
            {{
                "queries": ["query 1", "query 2"]
            }}
            """

            try:
                response = gemini(search_prompt)
                cleaned = response.replace("```json", "").replace("```", "").strip()
                queries = json.loads(cleaned)["queries"]
            except Exception:
                queries = [question]

            print("Agent: Searching Wikipedia...")

            search_results = {}

            for query in queries[:3]:
```

```
    try:
        results = wikipedia_search(query)

        for item in results:
            search_results[item["title"]] = item

    except Exception as e:
        print("Search warning:", e)

if not search_results:
    print("No Wikipedia articles found.")
    return

selected_titles = list(search_results.keys())[:5]

print("Agent: Retrieving articles...")

articles = []

for title in selected_titles:
    try:
        article = get_article(title)

        if article["content"].strip():
            articles.append(article)

    except Exception as e:
        print("Article retrieval warning:", e)

if not articles:
    print("Unable to retrieve article content.")
    return

context = ""

for article in articles:
    context += (
        "\n\nARTICLE: " + article["title"] +
        "\nURL: " + article["url"] +
        "\nCONTENT:\n" + article["content"][:10000]
    )

print("Agent: Generating answer...\n")

final_prompt = f"""
You are an intelligent Wikipedia Search Agent.

Answer the user's question using ONLY the supplied
Wikipedia article content.

User Question:
{question}

Retrieved Wikipedia Articles:
{context}

Instructions:
1. Provide a clear and accurate answer.
2. Organize the answer using headings and bullet points.
3. Include important facts and explanations.
4. Do not invent facts that are absent from the articles.
5. If information is insufficient, clearly mention it.
6. Mention which articles support the answer.
7. Do not invent references or URLs.
"""

answer = gemini(final_prompt)

print("=" * 65)
print("WIKIPEDIA SEARCH AGENT - FINAL ANSWER")
print("=" * 65)
print(answer)

print("\nSOURCES:")
for i, article in enumerate(articles, 1):
    print(f"{i}. {article['title']}")
    print(article["url"])

self.history.append({
    "question": question,
    "articles": [a["title"] for a in articles],
    "answer": answer
```

```
            })

            print("\nAgent execution completed successfully.")

    agent = WikipediaSearchAgent()

    question = input("Enter your question: ")
    agent.run(question)
```

```
Enter your Gemini API Key: ·····
Enter your question: who is hanuman

Agent: Understanding your query...
Agent: Searching Wikipedia...
Agent: Retrieving articles...
Agent: Generating answer...

================================================================
WIKIPEDIA SEARCH AGENT - FINAL ANSWER
================================================================
Based on the provided Wikipedia articles, here is who Hanuman is:

### Identity and Origins
* **Hindu Deity:** Hanuman is a deity in Hinduism, revered as a divine *vanara*. He is also known by other names
* **Divine Parentage:** He is traditionally believed to be the spiritual offspring or son of the celestial wind-g
  * In most Vaishnava traditions, he is considered the son and incarnation of Vayu.
  * In the Shaiva tradition, he is regarded as an incarnation of Shiva.
* **Immortality:** He is considered a *chiranjivi* (immortal being).

### Role and Significance in Hindu Epics
* **Companion of Lord Rama:** He is a central character in the *Ramayana* and is famous as a devoted companion an
* **Other Texts:** His tales are also recounted in the *Mahabharata* and various *Puranas*.
* **Qualities:** He is praised for his power, strength, courage, wisdom, celibacy (*brahmacharya*), and devotion

### Historical and Cultural Evolution
* **Development of Following:** Devotional practices centered on Hanuman were not prominent in early texts or arc
* **Nationalism and Defiance:** During the medieval Bhakti movement, figures like Samarth Ramdas portrayed him as
* **Literary Praise:** He is famously praised in Hindu devotional hymns such as the *Hanuman Chalisa*, written by

---
**Supporting Articles:**
* *Hanuman*
* *Hanuman Chalisa*

SOURCES:
1. Hanuman
```